

```

print("=== 3MTT DHF MINI PROJECT 2 FOR WEEK 3 (STEP 13 OF 89): MULTIPLE LINEAR REGRESSION EVALUATION MODEL ===")
print("Dr. Samuel Okhale 3MTT DHF AI/ML Multiple Regression Evaluation Mini Project 2 for Week 3")
print()
print("\n" + "="*60)
print("=== OUTPUT GENERATED ===")
print("\n" + "="*60)

# Step 1: Import necessary libraries for data manipulation and statistical analysis
print("Step 1: Import necessary libraries for data manipulation and statistical analysis")
import pandas as pd          # For data manipulation and analysis
import numpy as np           # For numerical operations
import matplotlib.pyplot as plt # For creating visualizations
import seaborn as sns        # For statistical data visualization
import statsmodels.api as sm  # For statistical modeling and regression analysis
from statsmodels.stats.outliers_influence import variance_inflation_factor # For VIF calculation
from scipy import stats       # For statistical tests and distributions
import warnings              # To handle warning messages
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output

# Set visualization style for better plots
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

# Step 2: Create a realistic Real-World marketing dataset
print("Step 2: Create a realistic Real-World marketing dataset")
np.random.seed(42) # Set seed for reproducible results

# Generate 200 data points for comprehensive analysis
n_samples = 200

# Create correlated marketing spend data to simulate real-world scenarios
# TV spending: Higher budget channel (20-100K)
tv_spend = np.random.normal(60, 20, n_samples)
tv_spend = np.clip(tv_spend, 20, 100) # Ensure realistic range

# Radio spending: Medium budget channel (10-50K), slightly correlated with TV
radio_spend = 0.3 * tv_spend + np.random.normal(15, 8, n_samples)
radio_spend = np.clip(radio_spend, 10, 50) # Ensure realistic range

# Social Media spending: Lower budget channel (5-30K), modern channel
social_spend = np.random.normal(18, 6, n_samples)
social_spend = np.clip(social_spend, 5, 30) # Ensure realistic range

# Create realistic sales data with different channel effectiveness
# TV has highest impact, Radio moderate, Social Media growing importance
sales = (2.5 * tv_spend +          # TV coefficient: $2.5K sales per $1K spend
         1.8 * radio_spend +       # Radio coefficient: $1.8K sales per $1K spend
         2.2 * social_spend +      # Social Media coefficient: $2.2K sales per $1K spend
         np.random.normal(0, 15, n_samples) + # Random noise
         50)                       # Base sales level

# Create DataFrame with realistic business context
marketing_data = pd.DataFrame({
    'TV_Spend_K': np.round(tv_spend, 1),          # TV spending in thousands
    'Radio_Spend_K': np.round(radio_spend, 1),    # Radio spending in thousands
    'Social_Spend_K': np.round(social_spend, 1),  # Social Media spending in thousands
    'Sales_K': np.round(sales, 1)                 # Sales in thousands
})

# Save dataset to CSV file for future use
marketing_data.to_csv('marketing_analysis_dataset.csv', index=False)
print("Dataset created and saved as 'marketing_analysis_dataset.csv'")
print(f"Dataset shape: {marketing_data.shape}")

# Step 3: Load dataset and perform initial exploratory data analysis
print("Step 3: Load dataset and perform initial exploratory data analysis")
# Load the dataset (demonstrating file loading process)
df = pd.read_csv('marketing_analysis_dataset.csv')

print("\n" + "="*60)
print("INITIAL EXPLORATORY DATA ANALYSIS")
print("="*60)

# Display basic dataset information
print("\n Dataset Overview:")

```

```

print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")

# Show first few rows to understand data structure
print("\n First 5 rows:")
print(df.head())

# Generate comprehensive descriptive statistics
print("\n Descriptive Statistics:")
print(df.describe().round(2))

# Check for missing values and data types
print("\n Data Quality Check:")
print("Missing values per column:")
print(df.isnull().sum())
print("\nData types:")
print(df.dtypes)

# Step 4: Test code with examples and verify expected outputs
print("Step 4: Test code with examples and verify expected outputs")
print("\n" + "="*60)
print("TESTING WITH SAMPLE DATA POINTS")
print("="*60)

# Display sample records to verify data quality
sample_indices = [0, 50, 100, 150] # Sample different parts of dataset
print("\n Sample data points for verification:")
for idx in sample_indices:
    row = df.iloc[idx]
    print(f"Row {idx}: TV=${row['TV_Spend_K']}K, Radio=${row['Radio_Spend_K']}K, Social=${row['Social_Spend_K']}K → Sales=${row['Sales_K']}K")

# Step 5: Create comprehensive visualizations
print("Step 5: Create comprehensive visualizations")
print("\n" + "="*60)
print("DATA VISUALIZATION AND ANALYSIS")
print("="*60)

# Create figure with multiple subplots
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Marketing Spend vs Sales Analysis', fontsize=16, fontweight='bold')

# Individual scatter plots for each channel
channels = ['TV_Spend_K', 'Radio_Spend_K', 'Social_Spend_K']
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']

for i, (channel, color) in enumerate(zip(channels, colors)):
    axes[0, i].scatter(df[channel], df['Sales_K'], alpha=0.6, color=color, s=50)
    axes[0, i].set_xlabel(f'{channel.replace("_", " ")}')
    axes[0, i].set_ylabel('Sales (K)')
    axes[0, i].set_title(f'{channel.replace("_Spend_K", "")} Spend vs Sales')
    axes[0, i].grid(True, alpha=0.3)

# Distribution plots
for i, channel in enumerate(channels):
    axes[1, i].hist(df[channel], bins=20, alpha=0.7, color=colors[i], edgecolor='black')
    axes[1, i].set_xlabel(f'{channel.replace("_", " ")}')
    axes[1, i].set_ylabel('Frequency')
    axes[1, i].set_title(f'Distribution of {channel.replace("_Spend_K", "")} Spend')
    axes[1, i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Correlation analysis
print("\n Correlation Matrix:")
correlation_matrix = df.corr()
print(correlation_matrix.round(3))

# Heatmap of correlations
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5, cbar_kws={"shrink": .8})
plt.title('Correlation Matrix: Marketing Channels and Sales', fontsize=14, fontweight='bold')
plt.show()

# Step 6: Build and evaluate multiple regression model
print("Step 6: Build and evaluate multiple regression model")
print("\n" + "="*60)

```

```

print("MULTIPLE REGRESSION ANALYSIS")
print("="*60)

# Prepare features and target variable
X = df[['TV_Spend_K', 'Radio_Spend_K', 'Social_Spend_K']]
y = df['Sales_K']

# Add constant term for intercept
X_with_const = sm.add_constant(X)

# Fit the regression model
model = sm.OLS(y, X_with_const).fit()

# Display comprehensive model results
print("\n Multiple Regression Model Results:")
print(model.summary())

# Step 7: Model Diagnostics and Validation (continued)
print("Step 7: Model Diagnostics and Validation (continued)")
print("\n" + "="*60)
print("MODEL DIAGNOSTICS AND VALIDATION")
print("="*60)

# Calculate predictions and residuals
predictions = model.predict(X_with_const)
residuals = y - predictions

# Model performance metrics
r_squared = model.rsquared
adj_r_squared = model.rsquared_adj
mse = np.mean(residuals**2)
rmse = np.sqrt(mse)
mae = np.mean(np.abs(residuals))

print(f"\n Model Performance Metrics:")
print(f"R-squared: {r_squared:.4f}")
print(f"Adjusted R-squared: {adj_r_squared:.4f}")
print(f"Root Mean Square Error (RMSE): {rmse:.2f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")

# Step 8: Multicollinearity Analysis
print("Step 8: Multicollinearity Analysis")
print("\n" + "="*60)
print("MULTICOLLINEARITY ANALYSIS")
print("="*60)

# Calculate Variance Inflation Factor (VIF)
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(X.columns))]

print("\n Variance Inflation Factor (VIF) Analysis:")
print(vif_data.round(2))
print("\nVIF Interpretation:")
print("• VIF < 5: Low multicollinearity")
print("• VIF 5-10: Moderate multicollinearity")
print("• VIF > 10: High multicollinearity (concern)")

# Step 9: Residual Analysis and Assumptions Testing
print("Step 9: Residual Analysis and Assumptions Testing")
print("\n" + "="*60)
print("RESIDUAL ANALYSIS AND ASSUMPTIONS TESTING")
print("="*60)

# Create residual plots
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Residual Analysis for Model Validation', fontsize=16, fontweight='bold')

# 1. Residuals vs Fitted Values
axes[0, 0].scatter(predictions, residuals, alpha=0.6, color='#FF6B6B')
axes[0, 0].axhline(y=0, color='black', linestyle='--', alpha=0.8)
axes[0, 0].set_xlabel('Fitted Values')
axes[0, 0].set_ylabel('Residuals')
axes[0, 0].set_title('Residuals vs Fitted Values')
axes[0, 0].grid(True, alpha=0.3)

```

```

# 2. Q-Q Plot for normality
stats.probplot(residuals, dist="norm", plot=axes[0, 1])
axes[0, 1].set_title('Q-Q Plot: Residual Normality')
axes[0, 1].grid(True, alpha=0.3)

# 3. Histogram of residuals
axes[1, 0].hist(residuals, bins=20, alpha=0.7, color='#4ECDC4', edgecolor='black')
axes[1, 0].set_xlabel('Residuals')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].set_title('Distribution of Residuals')
axes[1, 0].grid(True, alpha=0.3)

# 4. Scale-Location Plot
standardized_residuals = np.sqrt(np.abs(residuals / np.std(residuals)))
axes[1, 1].scatter(predictions, standardized_residuals, alpha=0.6, color='#45B7D1')
axes[1, 1].set_xlabel('Fitted Values')
axes[1, 1].set_ylabel('√|Standardized Residuals|')
axes[1, 1].set_title('Scale-Location Plot')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Statistical tests for assumptions
print("\n Statistical Tests for Model Assumptions:")

# Normality test (Shapiro-Wilk)
shapiro_stat, shapiro_p = stats.shapiro(residuals)
print(f"Shapiro-Wilk Normality Test:")
print(f"Statistic: {shapiro_stat:.4f}, p-value: {shapiro_p:.4f}")
print(f"Result: {'Residuals are normally distributed' if shapiro_p > 0.05 else 'Residuals may not be normally distributed'}")

# Step 10: Feature Importance and Coefficient Analysis
print("Step 10: Feature Importance and Coefficient Analysis")
print("\n" + "="*60)
print("FEATURE IMPORTANCE AND COEFFICIENT ANALYSIS")
print("="*60)

# Extract coefficients and create importance analysis
coefficients = model.params[1:] # Exclude intercept
p_values = model.pvalues[1:] # Exclude intercept
conf_intervals = model.conf_int()[1:] # Exclude intercept

# Create coefficient analysis DataFrame
coeff_analysis = pd.DataFrame({
    'Channel': ['TV', 'Radio', 'Social Media'],
    'Coefficient': coefficients.values,
    'P_Value': p_values.values,
    'Significant': p_values.values < 0.05,
    'CI_Lower': conf_intervals.iloc[:, 0].values,
    'CI_Upper': conf_intervals.iloc[:, 1].values
})

print("\n Coefficient Analysis:")
print(coeff_analysis.round(4))

# Visualize coefficient importance
plt.figure(figsize=(12, 8))
channels = coeff_analysis['Channel']
coeffs = coeff_analysis['Coefficient']
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']

bars = plt.bar(channels, coeffs, color=colors, alpha=0.7, edgecolor='black')
plt.xlabel('Marketing Channels')
plt.ylabel('Coefficient (Sales Impact per $1K Spend)')
plt.title('Marketing Channel Effectiveness: Coefficient Analysis')
plt.grid(True, alpha=0.3)

# Add value labels on bars
for bar, coeff in zip(bars, coeffs):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
             f'{coeff:.2f}', ha='center', va='bottom', fontweight='bold')

plt.show()

# Step 11: Scenario Analysis and Predictions (continued)

```

```

# Step 11: Scenario Analysis and Predictions (continued)
print("Step 11: Scenario Analysis and Predictions (continued)")
print("\n" + "="*60)
print("SCENARIO ANALYSIS AND PREDICTIONS")
print("="*60)

# Define different budget scenarios
scenarios = {
    'Current Average': [df['TV_Spend_K'].mean(), df['Radio_Spend_K'].mean(), df['Social_Spend_K'].mean()],
    'High TV Focus': [80, 20, 15],
    'Balanced Approach': [50, 35, 25],
    'Digital Focus': [40, 15, 30],
    'Budget Constraint': [30, 20, 10]
}

print("\n Marketing Budget Scenario Analysis:")
scenario_results = []

for scenario_name, spend_values in scenarios.items():
    # Create prediction input
    scenario_input = np.array([1] + spend_values) # Add constant term
    predicted_sales = model.predict(scenario_input)[0]
    total_spend = sum(spend_values)
    roi = (predicted_sales / total_spend) if total_spend > 0 else 0

    scenario_results.append({
        'Scenario': scenario_name,
        'TV_Spend': spend_values[0],
        'Radio_Spend': spend_values[1],
        'Social_Spend': spend_values[2],
        'Total_Spend': total_spend,
        'Predicted_Sales': predicted_sales,
        'ROI': roi
    })

    print(f"\n{scenario_name}:")
    print(f"   TV: ${spend_values[0]:.1f}K, Radio: ${spend_values[1]:.1f}K, Social: ${spend_values[2]:.1f}K")
    print(f"   Total Spend: ${total_spend:.1f}K")
    print(f"   Predicted Sales: ${predicted_sales:.1f}K")
    print(f"   ROI: {roi:.2f}x")

# Convert to DataFrame for better analysis
scenario_df = pd.DataFrame(scenario_results)

# Step 12: ROI and Efficiency Analysis
print("Step 12: ROI and Efficiency Analysis")
print("\n" + "="*60)
print("ROI AND EFFICIENCY ANALYSIS")
print("="*60)

# Calculate channel-specific ROI based on coefficients
channel_roi = {
    'TV': coefficients['TV_Spend_K'],
    'Radio': coefficients['Radio_Spend_K'],
    'Social Media': coefficients['Social_Spend_K']
}

print("\n Channel-Specific ROI Analysis:")
for channel, roi in channel_roi.items():
    print(f"{channel}: ${roi:.2f} sales return per $1 spent")

# Rank channels by effectiveness
roi_ranking = sorted(channel_roi.items(), key=lambda x: x[1], reverse=True)
print(f"\n Channel Effectiveness Ranking:")
for i, (channel, roi) in enumerate(roi_ranking, 1):
    print(f"{i}. {channel}: ${roi:.2f} return per $1 spent")

# Step 13: Statistical Significance and Confidence Intervals
print("Step 13: Statistical Significance and Confidence Intervals")
print("\n" + "="*60)
print("STATISTICAL SIGNIFICANCE ANALYSIS")
print("="*60)

print("\n Coefficient Significance Summary:")
for i, channel in enumerate(['TV', 'Radio', 'Social Media']):
    coeff = coefficients.iloc[i]
    p_val = p_values.iloc[i]
    ci_lower = conf_intervals.iloc[i, 0]

```

```

ci_upper = conf_intervals.iloc[i, 1]

significance = "****" if p_val < 0.001 else "***" if p_val < 0.01 else "**" if p_val < 0.05 else ""

print(f"\n{channel}:")
print(f"  Coefficient: {coeff:.3f} {significance}")
print(f"  P-value: {p_val:.4f}")
print(f"  95% CI: [{ci_lower:.3f}, {ci_upper:.3f}]")
print(f"  Interpretation: {'Statistically significant' if p_val < 0.05 else 'Not statistically significant'}")

# Step 14: Model Validation and Cross-Validation
print("Step 14: Model Validation and Cross-Validation")
print("\n" + "="*60)
print("MODEL VALIDATION")
print("="*60)

from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Prepare data for sklearn
X_sklearn = df[['TV_Spend_K', 'Radio_Spend_K', 'Social_Spend_K']]
y_sklearn = df['Sales_K']

# Create sklearn model for cross-validation
sklearn_model = LinearRegression()

# Perform 5-fold cross-validation
cv_scores = cross_val_score(sklearn_model, X_sklearn, y_sklearn, cv=5, scoring='r2')

print(f"\n 5-Fold Cross-Validation Results:")
print(f"Mean R2 Score: {cv_scores.mean():.4f} (±{cv_scores.std() * 2:.4f})")
print(f"Individual Fold Scores: {[f'{score:.4f}' for score in cv_scores]}")

# Step 15: COMPREHENSIVE SUMMARY REPORT
print("\n" + "="*80)
print("COMPREHENSIVE MARKETING ANALYSIS SUMMARY REPORT")
print("="*80)

```


=====

=== OUTPUT GENERATED ===

=====

Step 1: Import necessary libraries for data manipulation and statistical analysis
Step 2: Create a realistic Real-World marketing dataset
Dataset created and saved as 'marketing_analysis_dataset.csv'
Dataset shape: (200, 4)
Step 3: Load dataset and perform initial exploratory data analysis

=====

INITIAL EXPLORATORY DATA ANALYSIS

=====

Dataset Overview:
Shape: (200, 4)
Columns: ['TV_Spend_K', 'Radio_Spend_K', 'Social_Spend_K', 'Sales_K']

First 5 rows:

	TV_Spend_K	Radio_Spend_K	Social_Spend_K	Sales_K
0	69.9	38.8	8.4	324.7
1	57.2	36.7	14.4	276.9
2	73.0	45.6	18.0	367.1
3	90.5	50.0	18.3	426.7
4	55.3	20.6	15.3	265.2

Descriptive Statistics:

	TV_Spend_K	Radio_Spend_K	Social_Spend_K	Sales_K
count	200.00	200.00	200.00	200.00
mean	59.11	33.24	17.45	296.15
std	18.12	9.35	5.82	59.73
min	20.00	10.00	5.00	162.40
25%	45.92	26.40	13.20	256.08
50%	59.90	33.80	17.55	299.50
75%	70.00	39.78	21.42	332.42
max	100.00	50.00	30.00	433.40

Data Quality Check:
Missing values per column:
TV_Spend_K 0
Radio_Spend_K 0
Social_Spend_K 0
Sales_K 0
dtype: int64

Data types:
TV_Spend_K float64
Radio_Spend_K float64
Social_Spend_K float64
Sales_K float64
dtype: object
Step 4: Test code with examples and verify expected outputs

=====

TESTING WITH SAMPLE DATA POINTS

=====

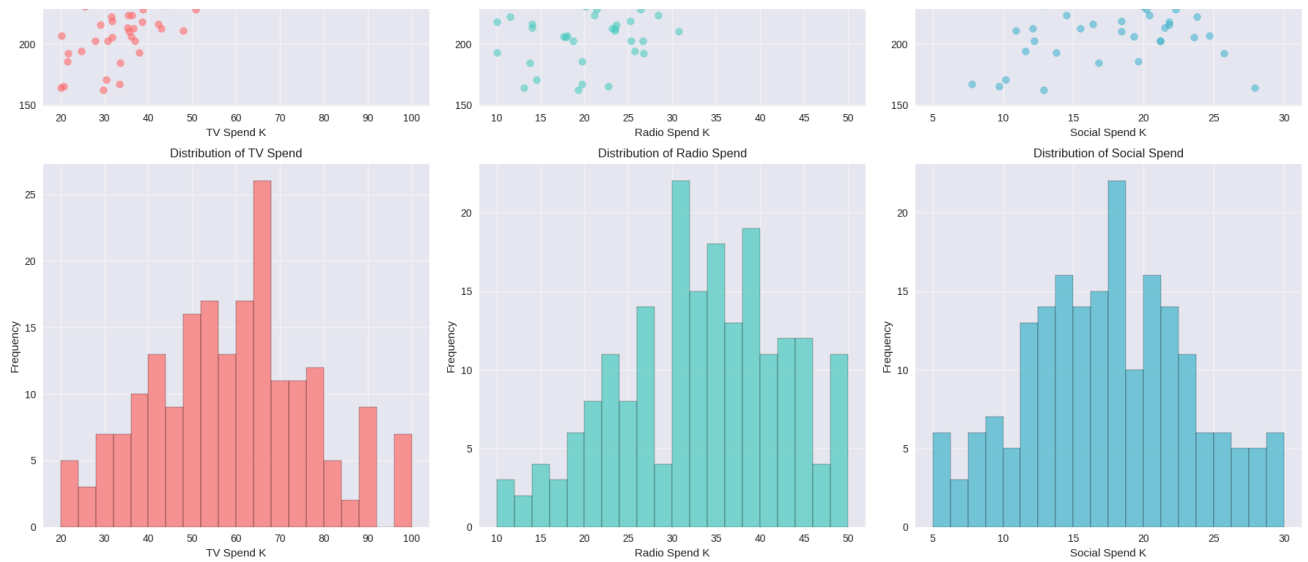
Sample data points for verification:
Row 0: TV=\$69.9K, Radio=\$38.8K, Social=\$8.4K → Sales=\$324.7K
Row 50: TV=\$66.5K, Radio=\$24.9K, Social=\$17.6K → Sales=\$327.5K
Row 100: TV=\$31.7K, Radio=\$17.9K, Social=\$23.6K → Sales=\$205.4K
Row 150: TV=\$65.0K, Radio=\$37.0K, Social=\$18.5K → Sales=\$306.0K
Step 5: Create comprehensive visualizations

=====

DATA VISUALIZATION AND ANALYSIS

=====

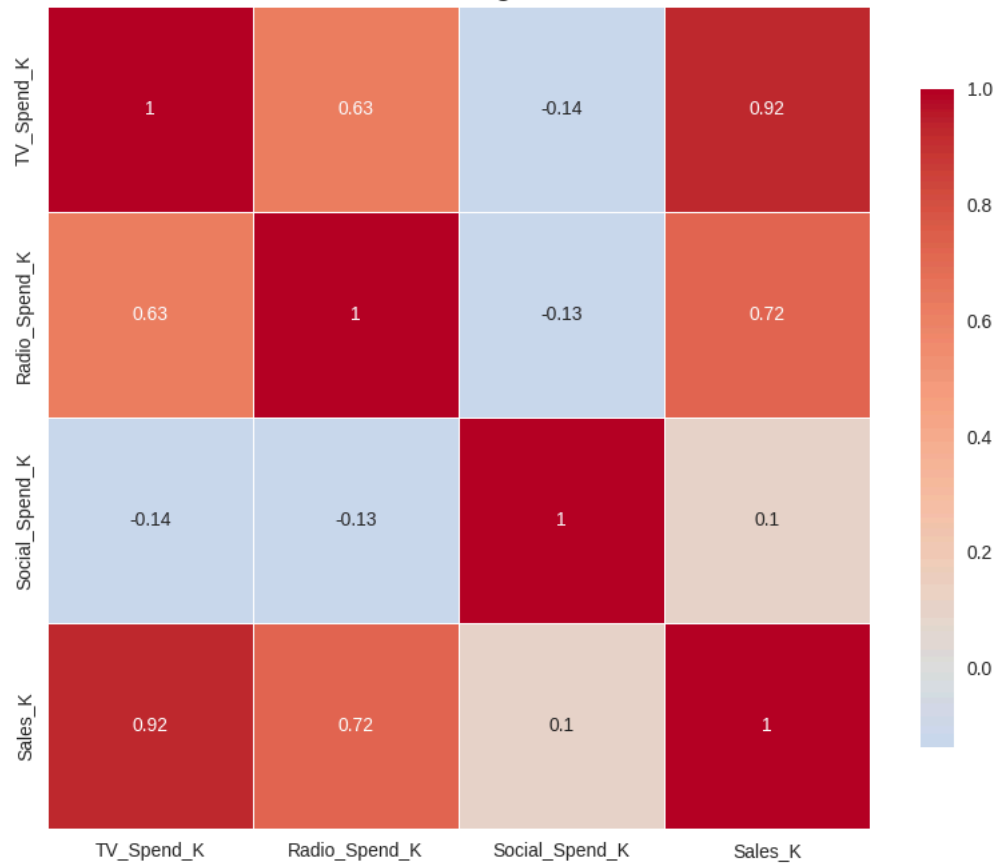




Correlation Matrix:

	TV_Spend_K	Radio_Spend_K	Social_Spend_K	Sales_K
TV_Spend_K	1.000	0.628	-0.137	0.921
Radio_Spend_K	0.628	1.000	-0.130	0.716
Social_Spend_K	-0.137	-0.130	1.000	0.100
Sales_K	0.921	0.716	0.100	1.000

Correlation Matrix: Marketing Channels and Sales



Step 6: Build and evaluate multiple regression model

MULTIPLE REGRESSION ANALYSIS

Multiple Regression Model Results:

OLS Regression Results

Dep. Variable:	Sales_K	R-squared:	0.936
Model:	OLS	Adj. R-squared:	0.935
Method:	Least Squares	F-statistic:	958.6
Date:	Sat, 27 Jun 2026	Prob (F-statistic):	8.15e-117
Time:	21:02:19	Log-Likelihood:	-826.07
No. Observations:	300	ATC:	1660

NO. OBSERVATIONS:	200	AIC:	1000.			
Df Residuals:	196	BIC:	1673.			
Df Model:	3					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	45.0230	5.735	7.851	0.000	33.714	56.332
TV_Spend_K	2.6376	0.077	34.432	0.000	2.487	2.789
Radio_Spend_K	1.5657	0.148	10.561	0.000	1.273	1.858
Social_Spend_K	2.4734	0.187	13.205	0.000	2.104	2.843
=====						
Omnibus:	0.052	Durbin-Watson:		2.102		
Prob(Omnibus):	0.974	Jarque-Bera (JB):		0.174		
Skew:	-0.006	Prob(JB):		0.917		
Kurtosis:	2.856	Cond. No.		386.		
=====						

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
Step 7: Model Diagnostics and Validation (continued)

=====

MODEL DIAGNOSTICS AND VALIDATION

=====

Model Performance Metrics:
R-squared: 0.9362
Adjusted R-squared: 0.9352
Root Mean Square Error (RMSE): 15.05
Mean Absolute Error (MAE): 11.99
Step 8: Multicollinearity Analysis

=====

MULTICOLLINEARITY ANALYSIS

=====

Variance Inflation Factor (VIF) Analysis:

	Feature	VIF
0	TV_Spend_K	17.69
1	Radio_Spend_K	19.01
2	Social_Spend_K	5.58

VIF Interpretation:

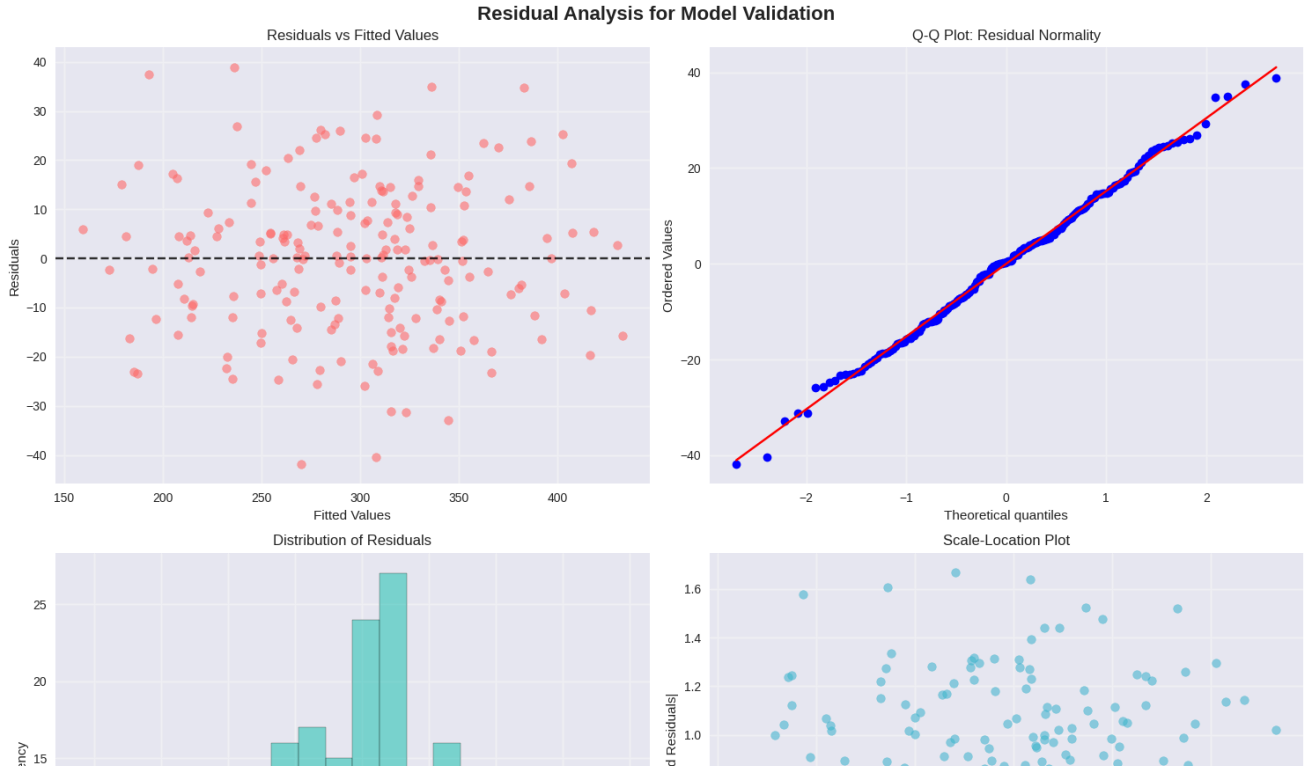
- VIF < 5: Low multicollinearity
- VIF 5-10: Moderate multicollinearity
- VIF > 10: High multicollinearity (concern)

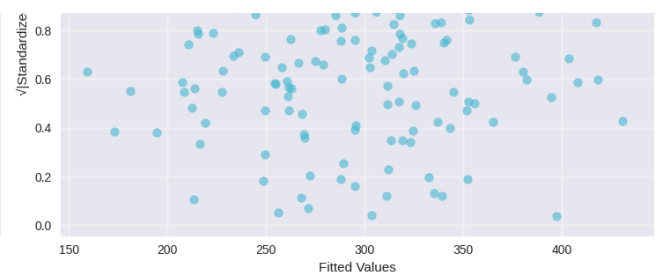
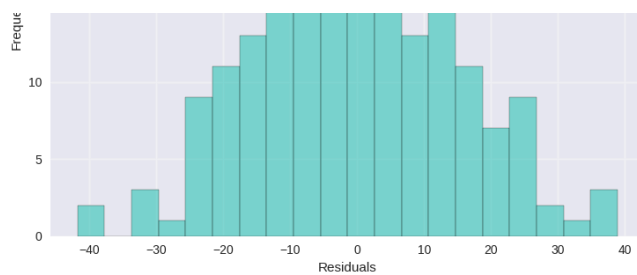
Step 9: Residual Analysis and Assumptions Testing

=====

RESIDUAL ANALYSIS AND ASSUMPTIONS TESTING

=====





Statistical Tests for Model Assumptions:
Shapiro-Wilk Normality Test:
Statistic: 0.9963, p-value: 0.9098
Result: Residuals are normally distributed
Step 10: Feature Importance and Coefficient Analysis

=====

FEATURE IMPORTANCE AND COEFFICIENT ANALYSIS

=====

Coefficient Analysis:

	Channel	Coefficient	P_Value	Significant	CI_Lower	CI_Upper
0	TV	2.6376	0.0	True	2.4865	2.7887
1	Radio	1.5657	0.0	True	1.2734	1.8581
2	Social Media	2.4734	0.0	True	2.1040	2.8428

Marketing Channel Effectiveness: Coefficient Analysis

